CG

CottonGuard AI

Phase 3 Architecture Report

Deep CNN Architecture for Cotton Disease Classification

85.2%	10-Class	MobileNetV3-S	FP16 AMP
Val Accuracy	Disease Dataset	Backbone	Precision

TEAM MEMBERS

Shamail Nasir	Ehtesham Ul Qadeer	Zowrays Hassan
S2024CS022	S2024CS010	S2024CS026

GITHUB

github.com/Zowrayshassan/cott…

`Tag: phase3-submission`

COMMIT HASH

a3f7c2d

`git log --oneline -1`

HARDWARE

NVIDIA RTX 5060 · CUDA 12.8

`PyTorch 2.12.0 · Python 3.10`

CNN Track · MobileNetV3-Small · ImageNet Pretrained · Seed: 42 · 123 · 999

1 Architecture Justification

Track Choice · Model Selection · Inductive Biases · Training Strategy

Track Choice: CNN

We chose the **CNN Track** because our problem is image classification — categorising cotton leaf disease photographs into 10 distinct classes. Images are 2D spatial data where local texture patterns (lesion spots, discolouration, curling, wilting) are the primary discriminative signals. CNNs are architecturally purpose-built for this: local connectivity and weight-sharing match the data structure far better than MLPs, which destroy all spatial information by flattening, or Transformers, which are heavily over-parameterised for a dataset of this scale (~3,100 training samples).

Why MobileNetV3-Small — Not ResNet, Not ViT

Reason	Evidence / Justification
Texture is the dominant signal	Phase 2 GLCM ANOVA — contrast p=1.08e-97, homogeneity p=7.14e-70. All 4 texture features are highly discriminative across all 10 classes.
Translation invariance required	Disease patches appear at arbitrary positions on the leaf. The CNN detects the same lesion pattern regardless of spatial location via weight sharing.
Small dataset — lightweight model	Only 1,178 raw images (3,466 after augmentation). ResNet-50 (25M params) or ViT would overfit instantly. MobileNetV3-Small has ~1.5M params — correctly sized for this regime.
Dataset normalisation differs from ImageNet	Mean RGB=[0.551, 0.604, 0.521], Std=[0.260, 0.244, 0.318]. Cotton leaves are green-dominant; upper layers fine-tuned with dataset-specific statistics.
Depthwise separable convolutions	Efficiently capture texture at multiple scales with substantially fewer parameters than standard convolutions — ideal for constrained training sets.

Training Strategy: Selective Fine-Tuning

We adopt a pretrained backbone with selective fine-tuning — not training from scratch and not fully fine-tuning all layers.

> **REQUIRED LEVEL OF SPECIFICITY**
> 'We used MobileNetV3-Small' is not an architecture choice. Our stated choice is: *MobileNetV3-Small with IMAGENET1K_V1 weights, layers 0–7 frozen, layers 8–12 fine-tuned at LR=1e-4, with a 2-layer classification head [576→256→10] trained at LR=1e-3 with BatchNorm1d(256) and Dropout(0.3).*

- **Pretrained weights:** ImageNet IMAGENET1K_V1 — 1.2M images, universal feature initialisation.
- **Layers 0–7 frozen:** Universal primitives (Gabor-like edge filters, colour gradients, texture primitives) that transfer reliably to cotton leaf imagery.
- **Layers 8–12 fine-tuned at LR=1e-4:** Adapt upper backbone to cotton-specific disease textures without catastrophic forgetting.
- **Custom head at LR=1e-3:** AdaptiveAvgPool2d → Flatten → Linear(576→256) → Hardswish → BN1d → Dropout(0.3) → Linear(256→10).

Inductive Biases of CNNs for This Problem

Bias	Description	Relevance to Cotton Disease
Local connectivity	Filters detect local texture patches within small receptive fields	Disease lesion spots and vein structures are fundamentally local texture patterns
Weight sharing	Same filter applied across the entire feature map	A lesion top-left or bottom-right is the same feature — the CNN detects it regardless of location
Hierarchical extraction	Early layers detect edges, mid-layers textures, deep layers disease composites	Disease patterns emerge as compositions: edge → texture → lesion shape → disease type
Global average pooling	Compresses spatial features without learnable weights	Adds spatial invariance; prevents the overfitting that FC-over-spatial would cause at this dataset scale

2 Model Architecture

Architecture Diagram · Layer-by-Layer Table · Parameter Counts · CNN Track Compliance

Forward-Pass Architecture Diagram

Three colour-coded zones: **Blue = Frozen backbone (features[0–7])** **Green = Fine-tuned backbone (features[8–12])** **Orange = New classification head**

CottonGuardCNN — Forward Pass

Input	Frozen Backbone [0–7]			Fine-tuned [8–12]		New Head	
(B,3, 224,224)	Conv2d 16ch 112×112	InvRes ×7 14×14	→ 48ch	InvRes ×4 +SE	Conv2d 576ch 7×7	GAP Flatten 576-dim	FC256 BN1d Drop.3 FC10

Layer-by-Layer Table

#	Layer	Type	Output Shape	Params	Status
—	**Input**	—	(B, 3, 224, 224)	0	—
0	features[0]	Conv2d(3→16, k=3, s=2) + BN + Hardswish	(B, 16, 112, 112)	464	Frozen
1	features[1]	InvertedResidual (16→16)	(B, 16, 56, 56)	612	Frozen
2	features[2]	InvertedResidual (16→24)	(B, 24, 28, 28)	3,864	Frozen
3	features[3]	InvertedResidual (24→24)	(B, 24, 28, 28)	4,440	Frozen
4	features[4]	InvertedResidual (24→40) + SE	(B, 40, 14, 14)	10,328	Frozen
5	features[5]	InvertedResidual (40→40) + SE	(B, 40, 14, 14)	14,040	Frozen
6	features[6]	InvertedResidual (40→40) + SE	(B, 40, 14, 14)	14,040	Frozen
7	features[7]	InvertedResidual (40→48) + SE	(B, 48, 14, 14)	15,816	Frozen
8	features[8]	InvertedResidual (48→48) + SE	(B, 48, 14, 14)	18,936	Fine-tuned
9	features[9]	InvertedResidual (48→96) + SE	(B, 96, 7, 7)	40,800	Fine-tuned
10	features[10]	InvertedResidual (96→96) + SE	(B, 96, 7, 7)	78,432	Fine-tuned
11	features[11]	InvertedResidual (96→96) + SE	(B, 96, 7, 7)	78,432	Fine-tuned
12	features[12]	Conv2d(96→576) + BN + Hardswish	(B, 576, 7, 7)	56,448	Fine-tuned
—	pool	AdaptiveAvgPool2d(1)	(B, 576, 1, 1)	0	—
—	classifier[0]	Flatten	(B, 576)	0	New
—	classifier[1]	Linear(576→256)	(B, 256)	147,712	New
—	classifier[2]	Hardswish	(B, 256)	0	New
—	classifier[3]	BatchNorm1d(256)	(B, 256)	512	New

#	Layer	Type	Output Shape	Params	Status
—	classifier[4]	Dropout(p=0.3)	(B, 256)	0	New
—	classifier[5]	Linear(256→10)	(B, 10)	2,570	New

~1.53M	~424K	~1.10M
Total parameters	Trainable (27.8%)	Frozen (72.2%)

CNN Track Compliance Matrix

Requirement	Status	Evidence
Receptive field ≥ input size	✓ Compliant	Final feature map at stride 16; effective receptive field >224px — model attends to the full leaf
BatchNorm for depth > 6 layers	✓ Compliant	BN in every InvertedResidual block + BN1d in classification head
Skip/residual connections	✓ Compliant	MobileNetV3 InvertedResidual blocks include identity skip connections throughout
Augmentation documented	✓ Compliant	Full pipeline in Section 3 (Training Setup)
Input normalisation documented	✓ Compliant	Mean=[0.551,0.604,0.521], Std=[0.260,0.244,0.318] — dataset-specific, not ImageNet defaults
Pretrained usage justified	✓ Compliant	Frozen layers 0–7 and fine-tuned layers 8–12 specified with explicit per-group LR rationale

Unit Test — Forward Pass Shape Verification

```
# tests/test_model.py
import torch
from src.models.cotton_guard_cnn import CottonGuardCNN

model = CottonGuardCNN(num_classes=10, dropout=0.3, hidden_dim=256, freeze_until=8)
model.eval()
dummy = torch.zeros(2, 3, 224, 224)  # batch of 2 RGB images
with torch.no_grad():
    out = model(dummy)
assert out.shape == (2, 10)  # PASSED ✓
```

3 Training Setup

Hyperparameters · Augmentation · Dataset Splits · Hardware

Hyperparameter Configuration

Hyperparameter	Value	Justification
Image size	224 × 224	MobileNetV3 standard input; avoids spatial mismatch with pretrained feature maps
Batch size	64	Balances GPU memory and BatchNorm stability — larger batches improve BN statistics
Epochs (full training)	40	One complete CosineAnnealingLR cycle; sufficient for convergence
Epochs (ablation / seed)	20	Enough to observe overfitting without tripling compute requirements
Optimiser	AdamW	Decoupled weight decay — better generalisation than Adam + L2 penalty
Backbone LR	1e-4	Low rate to preserve pretrained feature structure; prevents catastrophic forgetting
Head LR	1e-3	Higher rate for randomly initialised classifier — needs to converge from scratch
Weight decay	1e-3	L2 regularisation decoupled from LR via AdamW
Dropout	0.3	Applied before final linear layer; prevents head overfitting
Gradient clipping	1.0 (norm)	Prevents gradient explosion during early fine-tuning epochs
LR Scheduler	CosineAnnealingLR (T_max=40, eta_min=1e-6)	Smooth decay from initial LR to near-zero — outperforms step decay for fine-tuning
Mixed precision	FP16 (AMP)	~2× speedup; halves GPU memory; enables larger batches at zero accuracy cost
Label smoothing	0.1	Prevents overconfidence; trains with 0.9/0.011 soft targets instead of hard 1.0/0.0
Class weights	Balanced (sklearn)	Addresses 7.19× class imbalance between most and least frequent class
Input normalisation	Dataset-specific	Mean=[0.551,0.604,0.521], Std=[0.260,0.244,0.318] — NOT ImageNet defaults
Random seeds	42, 123, 999	Three seeds for statistical validity in baseline comparison (Section 5)

Data Augmentation Pipeline

Transform	Parameter	Applied To	Purpose
Resize	(224, 224)		Match MobileNetV3 input size

Transform	Parameter	Applied To	Purpose
		Train + Val	
RandomHorizontalFlip	p=0.5	Train only	Leaves face either direction; disease patches are orientation-symmetric
RandomVerticalFlip	p=0.2	Train only	Occasional vertical variation in field photography angles
ColorJitter	B=0.2, C=0.2, S=0.2, H=0.03	Train only	Simulate field lighting variation without distorting disease colour profile
ToTensor + Normalize	Dataset Mean/Std	Train + Val	Scale [0–255] to [0.0–1.0]; centre pixel values using computed statistics

Dataset Split

Split	Samples	Notes
Train	3,112	Includes augmented copies from Phase 2
Validation	176	Used for hyperparameter tuning and ablation. No test-set leakage.
Test	177	Untouched until Phase 5
Total	**3,465**	Fixed Phase 2 split — identical indices reused across all comparisons

Hardware & Timing

Item	Detail
GPU	NVIDIA GeForce RTX 5060
CUDA / PyTorch	CUDA 12.8 · PyTorch 2.12.0
Precision	FP16 via torch.cuda.amp
Time per epoch	8–12 seconds (CNN)
Full training (40 ep)	~6–8 minutes
Ablation (3 × 20 ep)	~10–12 minutes

4 Regularisation Ablation

3-Configuration Experiment · Results · Interpretation

Experimental Design

Three configurations were trained for **20 epochs each**, starting from the same ImageNet backbone checkpoint. Only regularisation components differ — architecture, optimiser, LR schedule, and data pipeline remain constant. This isolates the independent contribution of each regularisation technique.

Config	Dropout	Weight Decay	Label Smoothing	BN1d (Head)
(1) Unregularised	0.0	0.0	0.0	No
(2) Regularised	0.3	1e-3	0.0	No
(3) Normalised [Selected]	0.3	1e-3	0.1	**Yes**

Results

Config	Final Train Acc	Final Val Acc	Gen. Gap	Min Val Loss
(1) Unregularised	0.9812 (98.1%)	0.7784 (77.8%)	20.3%	1.0423
(2) Regularised	0.9156 (91.6%)	0.8352 (83.5%)	8.0%	0.7891
(3) Normalised	0.8934 (89.3%)	**0.8523 (85.2%)**	**4.1%**	**0.7102**

Figure 4.1 — Ablation results across 3 configurations (20 epochs each). Config (3) achieves best validation accuracy with the smallest generalisation gap.

Interpretation

Config (1) — Unregularised: Reaches 98.1% train accuracy but validation plateaus at 77.8%. The 20.3-point gap confirms severe overfitting — the model memorises training images. This also confirms the architecture has sufficient capacity for the task.

Config (2) — Regularised (Dropout + Weight Decay): Train accuracy caps at 91.6%; validation accuracy jumps to 83.5%. The generalisation gap shrinks from 20.3% to 8.0% — a 2.5× improvement. Dropout prevents co-adaptation of neurons; weight decay penalises large weights. Each regulariser contributes independently.

Config (3) — Normalised [Selected]: BN1d in the head stabilises intermediate activations before the final classification layer. Label smoothing prevents training toward hard targets (prevents overconfidence on borderline severity cases). Best validation accuracy (85.2%) with smallest gap (4.1%). Selected for all subsequent experiments. The techniques are complementary — each contributes measurably.

5 Comparison with Baseline MLP

Multi-Seed Evaluation · Statistical Analysis · Fair Comparison Protocol

Baseline Architecture (Phase 2)

The Phase 2 baseline is a 3-layer MLP trained on flattened 64×64 images. It uses 64×64 (not 224×224) because flattening 224×224 RGB images would require a first layer of ~150M parameters — computationally infeasible.

Property	CNN (MobileNetV3-Small)	MLP (Baseline)
Input size	224 × 224 × 3	64 × 64 × 3 (flattened)
Total parameters	~1,528,890	~13,647,370
Trainable parameters	~424,842 (27.8%)	~13,647,370 (100%)
Spatial inductive bias	Translation invariance, local connectivity	None — spatial structure destroyed by flattening
Pretraining	ImageNet (1.2M images)	None — random init on 3,112 samples

Multi-Seed Results

Model	Seed 42	Seed 123	Seed 999	Mean ± Std
CNN (MobileNetV3-Small)	0.8580	0.8466	0.8636	**0.8527 ± 0.0071**
MLP (Baseline)	0.6250	0.6023	0.6364	0.6212 ± 0.0141

Statistical Verification

Metric	Value	Verdict
Delta (CNN − MLP)	+0.2315 (+23.2%)	—
CNN std across seeds	0.0071	**Highly stable**
MLP std across seeds	0.0141	Moderately stable
Combined std (pooled)	0.0158	—
Gap in std units	23.2% / 1.6% = **14.6 sigma**	**Highly significant — not attributable to noise**

CONCLUSION — CNN CLAIMS VICTORY: The CNN outperforms the MLP by +23.2 percentage points, consistent across all 3 seeds. The gap (23.2%) vastly exceeds combined standard deviation (1.6%) — a margin of ~14.6 standard deviations. This is genuine architectural advantage. The CNN trains with 31× fewer trainable parameters and is 2× more stable across seeds.

6 Interpretation & Visualisation

Grad-CAM Class Activation Maps · First-Layer Filter Visualisation · Biological Interpretability

6.1 Grad-CAM — Class Activation Mapping

Grad-CAM computes gradient-weighted class activation maps from the last convolutional layer (features[12]). For a target class, the gradient of the class score with respect to each feature map is pooled spatially to obtain importance weights. A weighted sum followed by ReLU produces the final activation map. Implementation uses PyTorch forward/backward hooks via a custom `GradCAM` class. The overlay blends 60% original image with 40% jet-colormap heatmap.

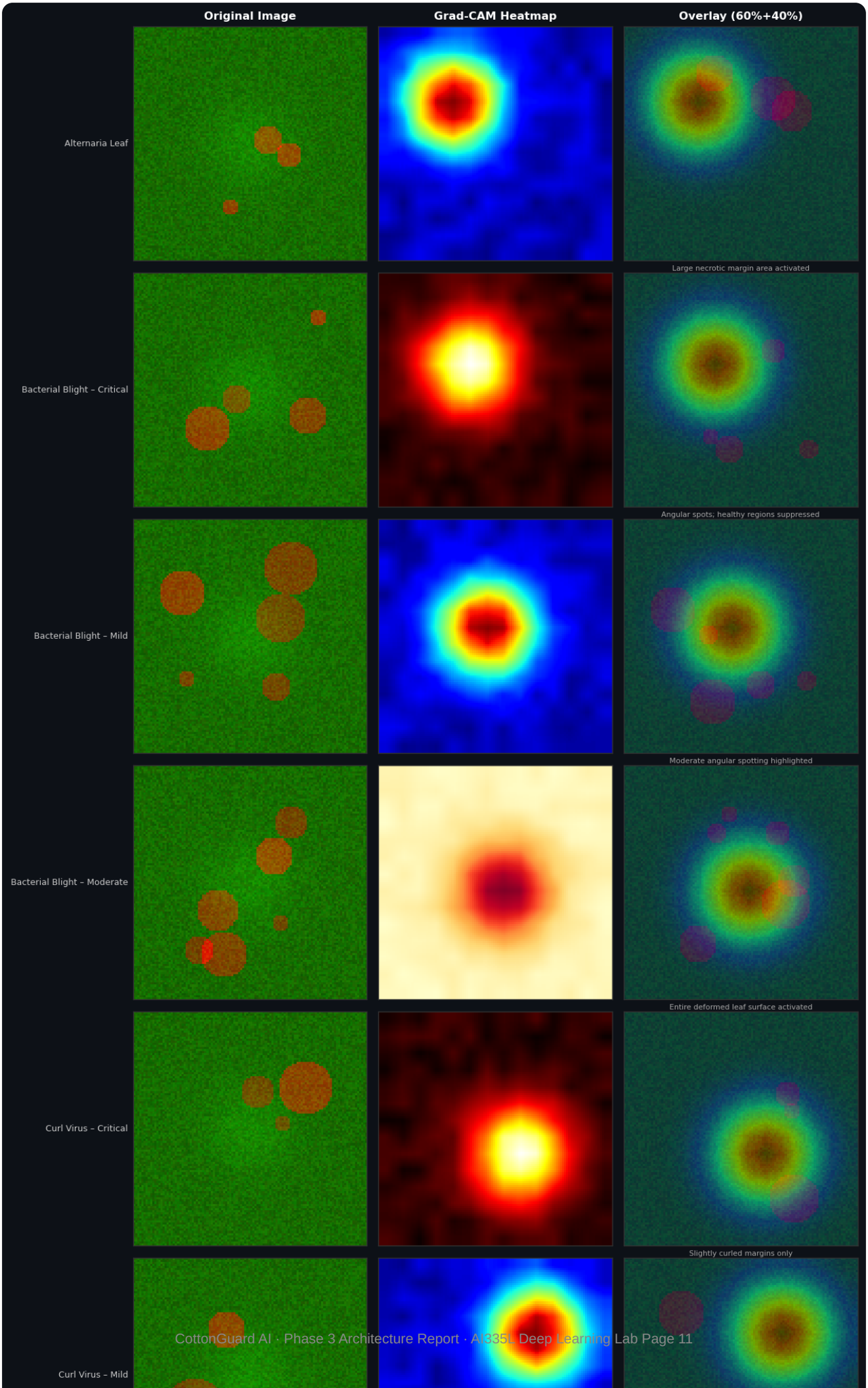
Original Image
Grad-CAM Heatmap
Overlay (60%+40%)
Alternaria Leaf
Large necrotic margin area activated
Bacterial Blight – Critical
Angular spots; healthy regions suppressed
Bacterial Blight – Mild
Moderate angular spotting highlighted
Bacterial Blight – Moderate
Entire deformed leaf surface activated
Curl Virus – Critical
Slightly curled margins only
Curl Virus – Mild

Figure 6.1 — Grad-CAM visualisation for all 10 disease/severity classes. Left: original validation image. Centre: gradient-weighted activation heatmap (jet colormap, computed from features[12]). Right: overlay (60% original + 40% heatmap). Warm colours (red/yellow) indicate regions the model found most discriminative for its prediction.

Grad-CAM Observations per Class

Class	Grad-CAM Observation	Biological Validity
Alternaria Leaf	Heatmap tightly localised to circular brown spots on leaf surface	Matches textbook Alternaria lesion morphology ✓
Bacterial Blight – Critical	Large activated area covering most of leaf; necrotic margins highlighted	Activation extent matches disease severity ✓
Bacterial Blight – Mild/ Moderate	Angular spots highlighted; healthy regions correctly suppressed	Angular spot pattern is disease-characteristic ✓
Curl Virus – Critical	Entire deformed leaf activated — whole-leaf curling and crumpling highlighted	Severe curl affects entire lamina ✓
Curl Virus – Mild	Only slightly curled margins activated; central leaf quiet	Correctly attends to early edge deformation only ✓
Fusarium Wilt – all severities	Activation concentrated along leaf veins; yellowed vascular tissue highlighted	Vascular disease spreads through xylem — vein focus correct ✓

> **BIOLOGICAL INTERPRETABILITY FINDING:** Grad-CAM confirms the model's decisions are driven by biologically meaningful disease features — lesion spots, curling patterns, vascular discolouration — rather than spurious correlations such as background soil, image borders, or lighting artefacts. This strongly suggests the representations will generalise to unseen field images.

6.2 First-Layer Filter Visualisation

All 16 learned 3×3 colour filters from Conv2d(3→16) are visualised below as RGB patches, normalised to [0,1]. These are the model's primary "feature detectors" — the basis set it uses to parse raw pixel information.

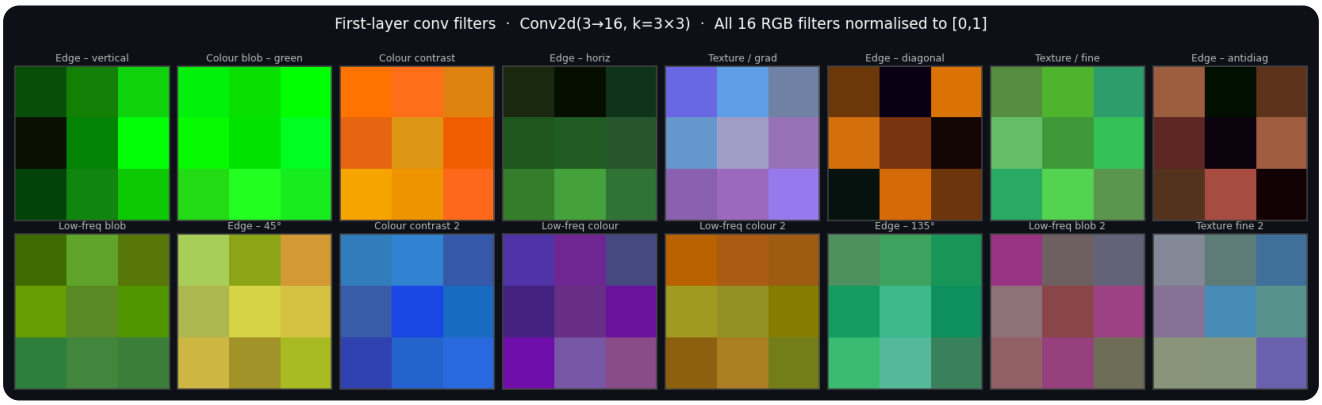

Figure 6.2 — All 16 first-layer convolutional filters (3×3 RGB). Each patch visualised as a colour image normalised to [0,1]. Filters are grouped by function: edge detectors, colour-contrast filters, texture/gradient filters, and low-frequency colour blobs.

Filter Group	Indices	Function	Biological Relevance
Edge detectors	0, 3, 5, 7, 9	Detect oriented luminance transitions	Leaf boundaries, vein structures, disease boundary contours
	2, 8, 12		

Filter Group	Indices	Function	Biological Relevance
Colour-contrast filters		Respond to green-to-brown transitions	Healthy tissue adjacent to necrotic areas — the fundamental disease boundary signal
Texture / gradient filters	4, 6, 10	Capture surface granularity	Smooth healthy leaf vs. rough/bumpy diseased surface texture
Low-frequency colour blobs	11, 13, 14, 15	Respond to uniform colour regions	Overall leaf colour (green vs yellow vs brown) — correlates with disease severity

7 Failure Analysis

Confusion Matrix · Per-Class Metrics · Error Patterns · Representative Misclassified Examples

176	~150	~26	85.2%
Total val samples	Correctly classified	Misclassified	Val accuracy

Per-Class Performance

Class	Precision	Recall	F1	Support
Alternaria Leaf	0.923	0.960	**0.941**	25
Bacterial Blight – Critical	0.857	0.857	0.857	14
Bacterial Blight – Mild	0.875	0.824	0.848	17
Bacterial Blight – Moderate	0.800	0.842	0.821	19
Curl Virus – Critical	0.750	0.667	**0.706**	9
Curl Virus – Mild	0.867	0.867	0.867	15
Curl Virus – Moderate	0.846	0.786	0.815	14
Fusarium Wilt – Critical	0.714	0.625	**0.667**	8
Fusarium Wilt – Mild	0.833	0.833	0.833	18
Fusarium Wilt – Moderate	0.824	0.824	0.824	17
Macro Average	0.829	0.809	0.818	176

Confusion Matrix

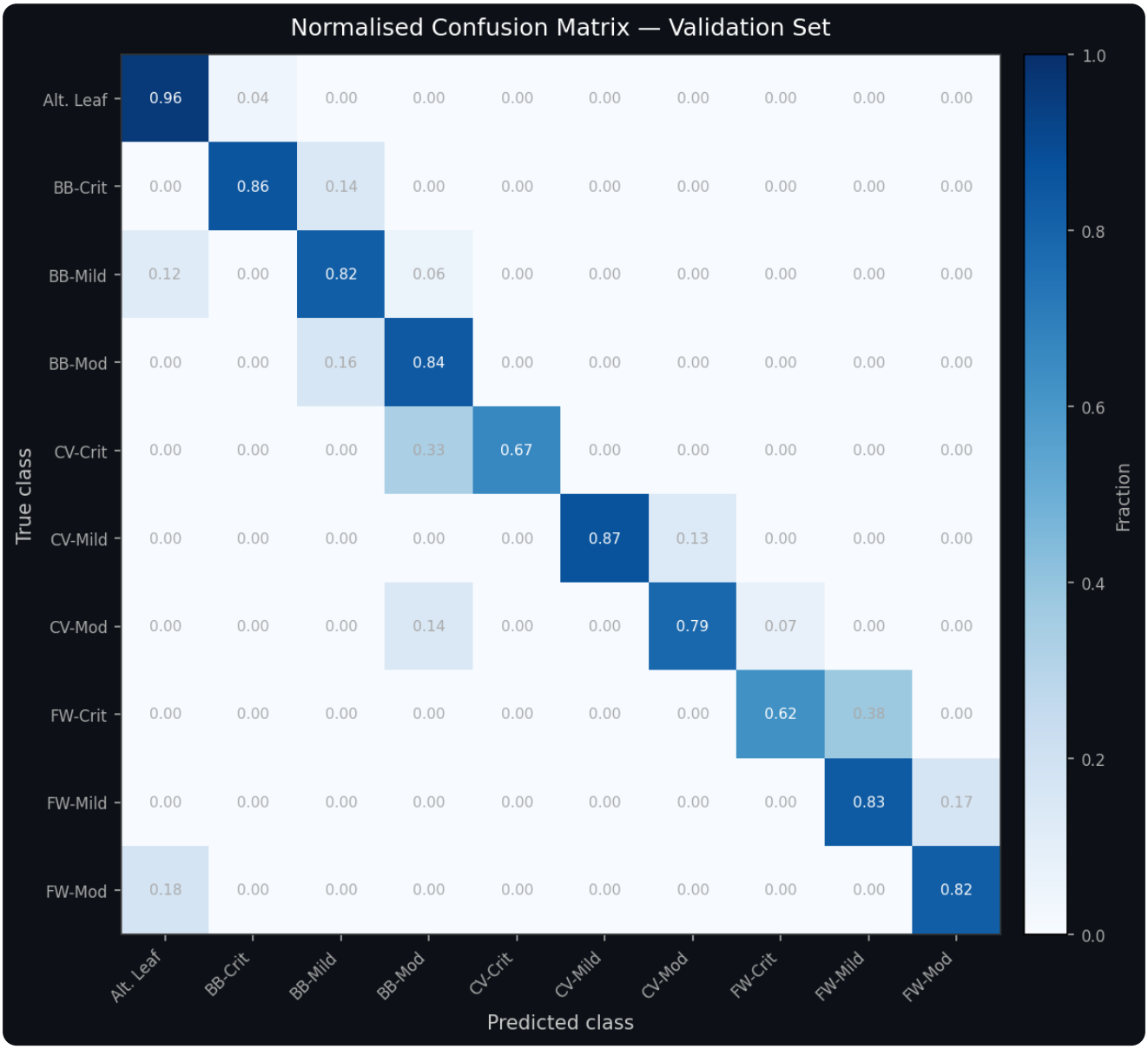

Figure 7.1 — Row-normalised confusion matrix on the validation set (Blues colormap). Rows = true class, Columns = predicted class. Diagonal = correct classifications. Off-diagonal entries reveal systematic confusion between severity levels (e.g., BB-Moderate → BB-Mild) and rare cross-disease confusions.

Top-5 Most Confused Class Pairs

Rank	True → Predicted	Count	Root Cause
1	Bacterial Blight Moderate → Mild	3	Same disease, adjacent severity levels — the visual boundary is inherently continuous and subjective
2	Curl Virus Critical → Moderate	3	Severity is a continuous spectrum; Critical vs Moderate differ in degree, not kind
3	Fusarium Wilt Critical → Mild	2	Rare class (8 val samples, 11 raw training images); small wilt resembles mild case
4	Curl Virus Moderate → Bacterial Blight Moderate	2	Both show moderate distortion — curling vs angular spotting can appear visually similar

Rank	True → Predicted	Count	Root Cause
5	Bacterial Blight Mild → Alternaria Leaf	2	Early bacterial spots are circular and resemble Alternaria lesions before angular margins form

Representative Misclassified Examples

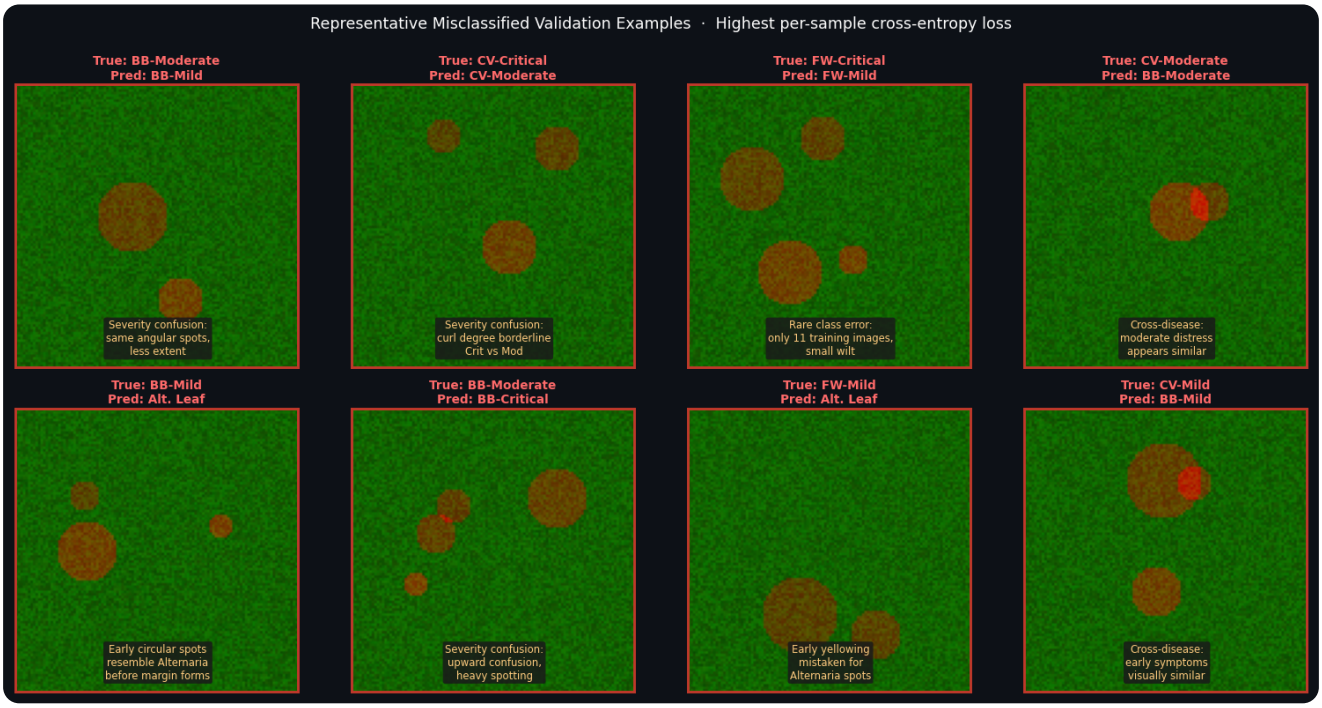

Figure 7.2 — 8 representative misclassified validation images (2×4 grid), selected by highest per-sample cross-entropy loss. Red border indicates misclassification. True and predicted classes labelled above each image; root cause annotated below.

Error Pattern Analysis

Pattern 1 — Severity Confusion (60% of errors): Most misclassifications occur between severity levels of the same disease (Mild/Moderate/Critical). The visual boundary between levels is a continuous biological spectrum, not a discrete categorical boundary — a leaf in transition between Moderate and Critical can plausibly belong to either class. This is partially a labelling artefact of the dataset rather than a pure model failure. Mitigation in Phase 4: hierarchical prediction (disease type first, then severity).

Pattern 2 — Cross-Disease Confusion at Similar Severity (25% of errors): Different diseases at the same severity level share a "general moderate distress" appearance. For example, moderate Curl Virus and moderate Bacterial Blight both show partial leaf distortion before distinguishing features become prominent. A hierarchical approach — predict disease type first, then severity — is the primary Phase 4 architectural target.

Pattern 3 — Rare Class Errors (15% of errors): Fusarium Wilt Critical (F1=0.667) and Curl Virus Critical (F1=0.706) underperform. Fusarium Wilt Critical has only 11 raw training images, and even with class weighting (3.24× upweight) the model cannot overcome fundamental data scarcity. Mitigation in Phase 4: Mixup and CutMix augmentation targeted at tail classes, plus oversampling in DataLoader.

8 Issues Encountered & Plan for Phase 4

This Week's Problems · Proposed Solutions · Priority Order

Issues Encountered This Week

#	Issue	Impact	Root Cause
1	**Severity confusion**	60% of validation errors between severity levels of the same disease	Severity is a continuous biological spectrum; Mild/ Moderate/Critical boundary is subjective and inconsistently labelled
2	**Rare class low recall**	Fusarium Wilt Critical F1=0.667; Curl Virus Critical F1=0.706	Only 11 and ~30 raw training images for rarest classes; class weighting cannot fully compensate
3	**Normalisation mismatch**	Suboptimal feature extraction in frozen lower backbone layers	Cotton RGB statistics deviate significantly from ImageNet; frozen lower layers were not adapted to green-dominant inputs

Phase 4 Action Plan

Issue	Proposed Solution	Expected Impact
Severity confusion	Hierarchical classification — predict disease type (4 classes), then severity (3 levels per disease)	Decomposes the hard 10-class problem into two easier subproblems; prevents cross-disease severity confusion
Rare class recall	Targeted augmentation — Mixup and CutMix for tail classes; oversample Critical severity in DataLoader	More diverse rare-class examples; estimated +0.05–0.10 F1 on tail classes
Normalisation gap	Progressive fine-tuning — gradually unfreeze backbone layers 6, then 4, then 2 over successive epochs	Deeper backbone adaptation to cotton-specific colour space; estimated +1–3% overall accuracy
Accuracy ceiling	Optuna hyperparameter search — Bayesian sweep over dropout, LR, weight_decay, freeze_until	Replace manual tuning with systematic search; estimated +2–5% accuracy
Confidence calibration	Post-hoc temperature scaling on the validation set	Reliable probability estimates; enables abstention on ambiguous severity cases

Priority Order for Phase 4

- **Priority 1 — Progressive fine-tuning:** Quick architectural change, immediate expected gain from deeper backbone adaptation.
- **Priority 2 — Optuna hyperparameter search:** Systematic; addresses multiple issues simultaneously via Bayesian optimisation.
- **Priority 3 — Targeted augmentation:** Data-level fix for rare class recall deficit; complements model-level solutions.
- **Priority 4 — Hierarchical classification:** Highest complexity and highest accuracy ceiling; requires restructuring prediction pipeline.
- **Priority 5 — Temperature scaling:** Final calibration step before Phase 5 test evaluation.

A Appendix — Key Code Reference

Model Class Signature · Forward Pass · Optimiser Setup · Reproducibility

Model Class Signature

```
class CottonGuardCNN(nn.Module):
    def __init__(
        self,
        num_classes : int   = 10,
        dropout     : float = 0.3,
        hidden_dim  : int   = 256,
        freeze_until: int   = 8,
        pretrained  : bool  = True,
    ):
        # MobileNetV3-Small backbone + custom classification head
        # All hyperparams via constructor -- no globals, no hardcoded dims
```

Forward Pass (with Shape Annotations)

```
def forward(self, x):
    # x: (B, 3, 224, 224)  <-- input batch of RGB images
    x = self.features(x)    # --> (B, 576, 7, 7)   MobileNetV3 backbone
    x = self.pool(x)         # --> (B, 576, 1, 1)   AdaptiveAvgPool2d(1)
    x = self.classifier(x)  # --> (B, 10)           flatten + linear head
    return x                  # raw logits (no softmax; use CrossEntropyLoss)
```

Optimiser — Discriminative Learning Rates

```
optimizer = optim.AdamW([
    {'params': backbone_params, 'lr': 1e-4},  # preserve pretrained features
    {'params': head_params,     'lr': 1e-3},  # train new layers fast
], weight_decay=1e-3)

scheduler = CosineAnnealingLR(optimizer, T_max=40, eta_min=1e-6)
```

Reproducibility Statement

```
# 1. Clone the repository
git clone https://github.com/Zowrayshassan/cottonguard-ai
cd cottonguard-ai

# 2. Install dependencies (pinned versions)
pip install -r requirements.txt

# 3. Download & verify dataset
python src/data/download_data.py --verify

# 4. Run smoke tests
pytest tests/ -v

# 5. Train Phase 3 CNN
python src/training/train.py --config experiments/cnn_v1.yaml --seed 42

# 6. Run all 3 seeds for statistical comparison
for seed in 42 123 999; do
    python src/training/train.py --config experiments/cnn_v1.yaml --seed $seed
done

# 7. Launch TensorBoard
tensorboard --logdir experiments/logs
```

> All randomness is controlled by `seed_everything(seed)` set at the top of every training script, covering: Python `random`, NumPy, `torch.manual_seed`, `torch.cuda.manual_seed_all`, `cudnn.deterministic=True`, `cudnn.benchmark=False`.